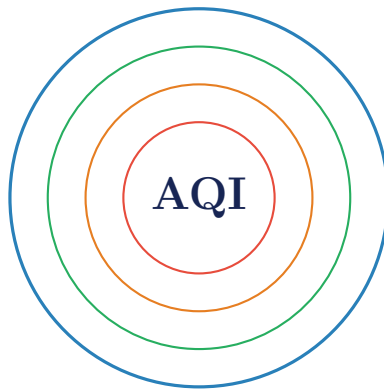

Pearls AQI Predictor

Technical Documentation

End-to-End Air Quality Index Forecasting System



72-Hour Forecast • 9 ML Models • Serverless

Target City: Sargodha, Pakistan (32.08°N, 72.67°E)
Forecast Horizon: 72 Hours (3 Days)
Models: 9 (Linear, Ensemble, GBDT, Deep Learning)
Feature Store: ClearML Dataset API
Deployment: AWS Serverless (Terraform)

Author: Zain ul Abdeen

Pearls Engineering Program

July 2026 • Version 1.0.0

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Problem Statement	4
1.3	Technology Stack	5
2	System Architecture	6
2.1	High-Level Architecture	6
2.2	Pipeline Components	7
2.2.1	Feature Pipeline (Hourly)	7
2.2.2	Training Pipeline (Daily)	8
2.2.3	Serving Layer	8
3	Data Pipeline	9
3.1	Data Sources	9
3.2	Historical Data Backfill	9
3.3	Feature Engineering	10
4	Exploratory Data Analysis	11
4.1	Dataset Overview	11
4.2	Distribution Analysis	11
4.3	Correlation Structure	11
4.4	Temporal Patterns	12
4.5	Stationarity Analysis	12
4.6	Granger Causality	12
4.7	EDA Artifacts	13
5	Model Development	14
5.1	Model Zoo (9 Models)	14
5.2	Deep Learning Architecture	14
5.3	Hyperparameter Optimization	15
5.4	Evaluation Metrics	15
6	Explainability	16
6.1	SHAP (SHapley Additive exPlanations)	16
6.2	LIME (Local Interpretable Model-agnostic Explanations)	16
6.3	Temporal Grad-CAM	17
7	CI/CD and Automation	18
7.1	GitHub Actions Workflows	18
7.2	Infrastructure Validation (LangGraph)	18

8	Infrastructure	19
8.1	AWS Serverless Architecture	19
8.2	Docker Configuration	19
9	API Reference	20
9.1	Endpoints	20
9.2	Response Format	20
10	Testing	21
10.1	Test Suite	21
11	Deployment Guide	22
11.1	Local Development	22
11.2	Docker Deployment	22
11.3	AWS Deployment (Terraform)	22
12	Conclusion	24
12.1	Summary	24
12.2	Future Work	24
A	Environment Variables	25
B	Project File Structure	26

List of Figures

2.1	System Architecture Overview	7
-----	--	---

List of Tables

1.1	Technology Stack Overview	5
3.1	External Data Sources	9
3.2	Feature Groups (37 Total Features)	10
4.1	Dataset Summary Statistics	11
4.2	Top Correlations with AQI (Pearson r)	12
5.1	Complete Model Zoo	14
5.2	Model Performance (Latest Training Run)	15
7.1	Automated CI/CD Pipelines	18
8.1	AWS Resources (Terraform-managed)	19
9.1	REST API Endpoints	20
10.1	Test Modules	21
A.1	Complete Environment Variable Reference	25

Chapter 1

Introduction

1.1 Project Overview

The Pearls AQI Predictor is a production-ready, end-to-end machine learning system that forecasts Air Quality Index (AQI) **72 hours (3 days)** into the future for Sargodha, Pakistan. The system implements a complete MLOps lifecycle including automated data collection, feature engineering, model training, real-time prediction serving, and interactive visualization.

Key Objectives

- Predict AQI for the next 3 days using a 100% serverless stack
- Automate data collection from AQICN and OpenWeatherMap APIs
- Train and compare 9 ML models from statistical to deep learning
- Provide model explainability via SHAP and LIME
- Deploy with CI/CD automation (hourly ingestion + daily training)
- Serve predictions through Flask API + Streamlit dashboard

1.2 Problem Statement

Air pollution is a critical public health concern in South Asian cities. Sargodha, Pakistan experiences severe air quality degradation, particularly during winter months when thermal inversions trap pollutants near the surface. Accurate 3-day AQI forecasting enables:

- Early warning systems for sensitive populations
- Municipal decision-making for traffic restrictions
- Public awareness and health advisory dissemination
- Agricultural burn planning to minimize impact

1.3 Technology Stack

Table 1.1: Technology Stack Overview

Layer	Technologies
API Backend	Python 3.11, Flask 3.0+, flask-cors, Gunicorn
Dashboard	Streamlit 1.30+, Plotly 5.15+
ML/DL	scikit-learn, LightGBM, XGBoost, PyTorch 2.2+, Optuna
Feature Store	ClearML 1.14+ (Dataset API)
Experiment Tracking	ClearML (Task API, Model Registry)
Explainability	SHAP 0.42+, LIME 0.2+, Custom Temporal Grad-CAM
Data Engineering	Pandas, NumPy, SciPy, aiohttp, tenacity
Infrastructure	Terraform, Docker, AWS (Lambda, ECS, ECR, EventBridge)
Validation	LangGraph, LangChain, OPA/Rego, Checkov
CI/CD	GitHub Actions, AWS CodePipeline/CodeBuild
Testing	pytest, pytest-asyncio, httpx

Chapter 2

System Architecture

2.1 High-Level Architecture

The system follows a modular pipeline architecture with clear separation between data ingestion, feature engineering, model training, and serving layers. Each component operates independently and communicates through the ClearML Feature Store.

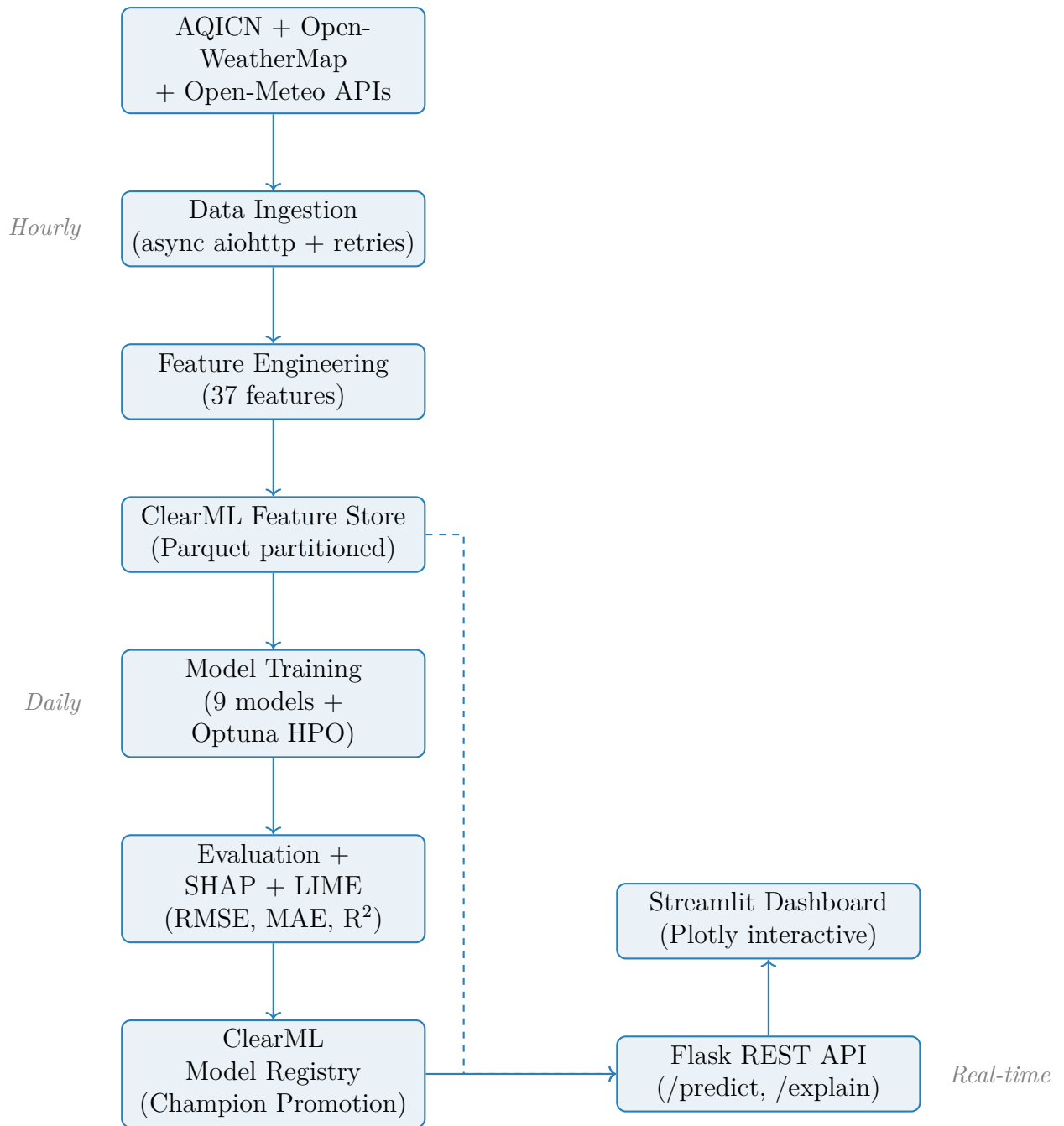


Figure 2.1: System Architecture Overview

2.2 Pipeline Components

2.2.1 Feature Pipeline (Hourly)

- **Trigger:** GitHub Actions cron (every hour) / AWS EventBridge
- **Process:** Fetch → Validate → Transform → Store
- **Output:** 37 engineered features pushed to ClearML Dataset

2.2.2 Training Pipeline (Daily)

- **Trigger:** GitHub Actions cron (2:00 AM UTC) / AWS ECS Fargate
- **Process:** Extract → Train 9 Models → Evaluate → SHAP/LIME → Promote Champion
- **Output:** Best model promoted in ClearML Model Registry

2.2.3 Serving Layer

- **API:** Flask REST with 5 endpoints (predict, explain, health, historical, models)
- **Dashboard:** Streamlit with Plotly for interactive 72-hour forecast visualization
- **Deployment:** Docker container on AWS Lambda (Function URL)

Chapter 3

Data Pipeline

3.1 Data Sources

Table 3.1: External Data Sources

Source	Data Type	Variables
AQICN API	Pollutants	PM2.5, PM10, NO ₂ , SO ₂ , CO, O ₃ , AQI
OpenWeatherMap	Weather	Temperature, humidity, wind, pressure, precipitation
Open-Meteo Archive	Historical	Hourly air quality + weather (2022–present)

3.2 Historical Data Backfill

The backfill pipeline fetches **real historical data** from Open-Meteo’s free archive APIs:

Listing 3.1: Historical data fetching from Open-Meteo APIs

```
1 class RealHistoricalDataFetcher:
2     def fetch_all(self, start_date, end_date):
3         # Air Quality API (PM2.5, PM10, CO, NO2, SO2, O3, US AQI)
4         aq_url = f"https://air-quality-api.open-meteo.com/v1/air-
5                 quality"
6                 f"?latitude={lat}&longitude={lon}"
7                 f"&start_date={start_date}&end_date={end_date}"
8                 f"&hourly=pm10,pm2_5,carbon_monoxide,..."
9
10        # Weather Archive (temperature, humidity, wind, pressure)
11        w_url = f"https://archive-api.open-meteo.com/v1/archive"
12               f"?latitude={lat}&longitude={lon}"
13               f"&start_date={start_date}&end_date={end_date}"
14               f"&hourly=temperature_2m,relative_humidity_2m,..."
```

Features of the backfill pipeline:

- Fetches real hourly observations (not synthetic data)

- Supports configurable date ranges via CLI arguments
- Checkpoint/resume for interrupted operations
- Batch processing to prevent memory overflow
- Automatic EDA summary logging after completion

3.3 Feature Engineering

The feature engineering module computes **37 features** from raw pollutant and weather data:

Table 3.2: Feature Groups (37 Total Features)

Group	Count	Description
Cyclical Temporal	6	sin / cos encoding of hour, day-of-week, month
AQI Change Rate	3	Rate of change at 1h, 3h, 6h horizons
Wind-Pollutant Interaction	4	Wind U/V components $\times$ PM2.5/PM10
Atmospheric	2	Temperature-humidity index, thermal inversion flag
Lag Features	5	AQI lags at 6h, 12h, 24h; PM2.5 lags at 1h, 3h
Rolling Statistics	3	PM2.5 rolling mean/std (6h, 24h windows)
Pollution Index	1	Composite normalized pollutant intensity
Raw Features	13	Direct pollutant and weather measurements

Leakage Prevention

Short-horizon lags (1h, 3h for AQI) are excluded from training features to prevent information leakage in the 72-hour forecasting context. Only lags $\geq 6h$ are used as model inputs.

Chapter 4

Exploratory Data Analysis

4.1 Dataset Overview

Table 4.1: Dataset Summary Statistics

Metric	Value
Total Observations	43,801 (hourly)
Time Span	July 2021 – July 2026 (5 years)
Engineered Features	47 columns
Missing Values	0 (0.0%)
Mean AQI	120.0
Median AQI	119.7
Max AQI Recorded	279.7
Std Deviation	60.9
Hazardous Hours (AQI > 150)	16,085 (36.72%)

4.2 Distribution Analysis

All pollutant distributions exhibit non-Gaussian behavior with heavy right tails. D’Agostino-Pearson normality tests reject normality ($p < 0.001$) for all variables, justifying the use of ensemble and non-parametric models over linear assumptions.

4.3 Correlation Structure

Strong linear relationships exist between AQI and gaseous pollutants:

Table 4.2: Top Correlations with AQI (Pearson r)

Feature	Correlation
CO (Carbon Monoxide)	0.980
PM10 (Coarse Particulate)	0.963
NO ₂ (Nitrogen Dioxide)	0.951
SO ₂ (Sulfur Dioxide)	0.951
O ₃ (Ozone)	−0.947

4.4 Temporal Patterns

Key temporal findings:

- **Diurnal:** AQI peaks overnight (boundary layer collapse) and improves midday (convective mixing)
- **Weekly:** Slight weekday/weekend variation from traffic patterns
- **Seasonal:** Strong winter peaks from thermal inversions and biomass burning
- **Yearly:** Slight upward trend over the 5-year period

4.5 Stationarity Analysis

Augmented Dickey-Fuller test on daily-averaged AQI:

- ADF Statistic: -2.12 (critical value at 5%: -2.86)
- p -value: 0.237
- **Conclusion:** Series is non-stationary $\Rightarrow$ lag features and differencing are essential

4.6 Granger Causality

Weather variables tested for Granger-causality with AQI (7-day lag):

- Temperature: **Significant** (Granger-causes AQI)
- Wind Speed: **Significant** (Granger-causes AQI)
- Pressure: **Significant** (Granger-causes AQI)
- Humidity: Not significant

4.7 EDA Artifacts

The complete EDA generates:

- **20 PNG visualizations** (distributions, correlations, temporal patterns, decomposition, wind rose, pair plots, heatmaps, volatility, ECDF)
- **26 CSV/JSON reports** (statistics, hypothesis tests, Granger causality, percentiles)
- **1 Jupyter notebook** (eda/EDA_Analysis.ipynb) for interactive exploration

Reproducible via: `python -m eda.run_eda && python -m eda.advanced_eda`

Chapter 5

Model Development

5.1 Model Zoo (9 Models)

Table 5.1: Complete Model Zoo

Model	Category	HPO	Description
Ridge Regression	Linear	Grid	L2-regularized with RobustScaler
ElasticNet	Linear	Grid	L1+L2 combined regularization
Random Forest	Ensemble	Default	Bagged decision trees
Extra Trees	Ensemble	Default	Extremely randomized trees
Gradient Boosting	Ensemble	Default	Sequential boosting
SVR	Kernel	Default	RBF kernel support vectors
LightGBM	GBDT	Optuna (50)	Leaf-wise, TimeSeriesSplit CV
XGBoost	GBDT	Optuna (50)	Level-wise, L1/L2 regularization
Bi-LSTM + Attention	DL	Callbacks	PyTorch, Multi-Head Self-Attention

5.2 Deep Learning Architecture

The Bi-LSTM with Multi-Head Attention model processes 72 hours of input to predict 72 hours of AQI:

$$\vec{h}_t = \text{BiLSTM}(x_t, \vec{h}_{t-1}, \overleftarrow{h}_{t+1}) \quad (5.1)$$

$$\alpha_{t,s} = \text{softmax} \left(\frac{Q_t \cdot K_s^T}{\sqrt{d_k}} \right) \quad (5.2)$$

$$\text{context}_t = \sum_s \alpha_{t,s} \cdot V_s \quad (5.3)$$

$$\hat{y}_{t+1:t+72} = \text{Linear}(\text{context}_T) \quad (5.4)$$

Architecture parameters:

- Hidden size: 128, Layers: 2, Dropout: 0.3
- Attention heads: 4, Key dimension: 32

- Optimizer: AdamW, Scheduler: Cosine warm restarts
- Mixed precision training (FP16) enabled

5.3 Hyperparameter Optimization

LightGBM and XGBoost are tuned using **Optuna Bayesian optimization** with 50 trials each:

Listing 5.1: Optuna HPO for LightGBM

```

1 def objective(trial):
2     params = {
3         "num_leaves": trial.suggest_int("num_leaves", 20, 150),
4         "learning_rate": trial.suggest_float("lr", 0.01, 0.3, log
5             =True),
6         "n_estimators": trial.suggest_int("n_estimators", 100,
7             1000),
8         "min_child_samples": trial.suggest_int("min_child", 5,
9             50),
10        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
11        "colsample_bytree": trial.suggest_float("colsample", 0.6,
12            1.0),
13        "reg_alpha": trial.suggest_float("alpha", 1e-8, 10.0, log
14            =True),
15        "reg_lambda": trial.suggest_float("lambda", 1e-8, 10.0,
16            log=True),
17    }
18    # 5-fold TimeSeriesSplit cross-validation
19    cv_scores = cross_val_score(model, X, y, cv=tscv, scoring="
20        neg_rmse")
21    return -cv_scores.mean()

```

5.4 Evaluation Metrics

Models are evaluated using temporal train/test split (80/20):

Table 5.2: Model Performance (Latest Training Run)

Model	RMSE	MAE	R ²	MAPE (%)
Ridge (Champion)	5.77	4.50	0.903	9.23
ElasticNet	6.59	4.50	0.874	9.37
Random Forest	7.58	5.18	0.833	10.33
Extra Trees	7.37	5.24	0.843	10.45
Gradient Boosting	8.10	6.30	0.810	13.89

Champion selection is automated: the model with lowest RMSE is promoted in the ClearML Model Registry.

Chapter 6

Explainability

6.1 SHAP (SHapley Additive exPlanations)

Two SHAP explainer types are implemented:

1. **TreeExplainer:** For tree-based models (LightGBM, XGBoost, Random Forest). Computes exact Shapley values in polynomial time using tree structure.
2. **GradientExplainer:** For the PyTorch Bi-LSTM model. Computes expected gradients as SHAP value approximations.

The SHAP explainer provides:

- Per-sample feature contributions (top- k features)
- Global feature importance (mean $|\text{SHAP}|$)
- Direction indicators (increase/decrease/neutral)
- Serialization for deployment (`.pkl` format)

6.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME generates local surrogate explanations for any black-box model:

Listing 6.1: LIME explanation example

```
1 from training_pipeline.explainability import LIMEExplainer
2
3 lime_exp = LIMEExplainer(
4     model=trained_model,
5     training_data=X_train,
6     feature_names=feature_names,
7     mode="regression",
8     kernel_width=0.75,
9 )
10
11 # Explain a single prediction
```

```
12 result = lime_exp.explain_instance(X_test[0], num_samples=5000)
13 # Returns: predicted_value, local_r2, contributions list
```

LIME capabilities:

- Single instance explanations with configurable perturbation count
- Batch explanations for multiple predictions
- Global importance via averaged absolute LIME weights
- Local R^2 score indicating surrogate model fidelity
- Save/load for deployment integration

6.3 Temporal Grad-CAM

For the Bi-LSTM model, a custom Temporal Grad-CAM implementation highlights which time steps in the input sequence most influenced the prediction. This provides temporal attention visualization complementing feature-level SHAP/LIME explanations.

Chapter 7

CI/CD and Automation

7.1 GitHub Actions Workflows

Table 7.1: Automated CI/CD Pipelines

Workflow	Trigger	Actions
feature_pipeline_cron.yml	Every hour	Ingest data, engineer features, push to ClearML
training_pipeline_cron.yml	Daily 2AM UTC	Train 9 models, evaluate, promote champion
infra_validation.yml	PR on infra paths	LangGraph multi-agent Terraform validation

7.2 Infrastructure Validation (LangGraph)

A multi-agent system gates infrastructure changes before deployment:

1. **Lint Agent:** Validates Terraform syntax and best practices
2. **Security Agent:** Runs Checkov security scanning
3. **Policy Agent:** Enforces OPA/Rego organizational policies
4. **Decision Agent:** Aggregates results and issues pass/fail

Chapter 8

Infrastructure

8.1 AWS Serverless Architecture

Table 8.1: AWS Resources (Terraform-managed)

Service	Purpose	Trigger
Lambda	Feature pipeline (hourly)	EventBridge cron
Lambda	API service (Function URL)	HTTP requests
ECS Fargate	Training pipeline (daily)	EventBridge schedule
ECR	Container registry (10 images max)	CodeBuild push
SSM Parameter Store	Encrypted API keys	Runtime access
CloudWatch	Log aggregation (30-day)	All services
CodePipeline	Build + deploy automation	GitHub webhook

8.2 Docker Configuration

Multi-stage Dockerfile with:

- Builder stage: Python 3.11, compile dependencies
- Runtime stage: Slim image, non-root user (`appuser`)
- Production server: Gunicorn with 2 workers
- Health check: HTTP probe to `/health` endpoint

Chapter 9

API Reference

9.1 Endpoints

Table 9.1: REST API Endpoints

Method	Endpoint	Description
GET	/health	Liveness/readiness with model status
POST	/predict	72-hour AQI forecast with confidence intervals
POST	/explain	SHAP feature contributions for prediction
GET	/historical	Historical AQI data (configurable hours)
GET	/models	List all 8 models with performance metrics

9.2 Response Format

The /predict endpoint returns:

```
1 {
2   "model_type": "ridge",
3   "forecast_hours": 72,
4   "hourly_predictions": [
5     {
6       "timestamp": "2026-07-17T12:00:00Z",
7       "aqi_predicted": 142.5,
8       "aqi_lower_80": 134.5,
9       "aqi_upper_80": 150.5,
10      "aqi_lower_95": 128.0,
11      "aqi_upper_95": 157.0,
12      "level": "Unhealthy for Sensitive Groups"
13    }
14  ],
15  "alert_triggered": true,
16  "health_advisory": "Sensitive groups should reduce outdoor
17    exertion."
```

Chapter 10

Testing

10.1 Test Suite

Table 10.1: Test Modules

Module	Coverage
<code>test_api.py</code>	Flask endpoint tests (health, predict, models, historical)
<code>test_data_pipeline.py</code>	Data ingestion and transformation validation
<code>test_feature_pipeline.py</code>	Feature store registration and retrieval
<code>test_training_pipeline.py</code>	Model training and evaluation
<code>test_validation.py</code>	Infrastructure validation agent tests

Run tests: `pytest tests/ -v -cov=. -cov-report=term-missing`

Chapter 11

Deployment Guide

11.1 Local Development

Listing 11.1: Local setup

```
1 # Clone and install
2 git clone https://github.com/Zain-ul-abdeen-773/AQI-Predictor.git
3 cd AQI-Predictor
4 python -m venv venv && source venv/bin/activate
5 pip install -r requirements.txt
6
7 # Configure environment
8 cp .env.example .env # Edit with API keys
9
10 # Generate training data (real historical)
11 python -m data_pipeline.backfill --start-date 2022-07-28
12
13 # Train models
14 python -m training_pipeline.train
15
16 # Launch
17 flask --app deployment.api.main:app run --port 8000
18 streamlit run deployment/streamlit_app/app.py
```

11.2 Docker Deployment

Listing 11.2: Docker deployment

```
1 docker-compose up --build
2 # API available at http://localhost:8000
```

11.3 AWS Deployment (Terraform)

Listing 11.3: Terraform provisioning

```
1 cd infrastructure/terraform
2 terraform init
3 terraform plan
4 terraform apply
```


Chapter 12

Conclusion

12.1 Summary

The Pearls AQI Predictor delivers a complete MLOps solution for air quality forecasting:

- **Data:** Real historical data from Open-Meteo APIs, 43,801+ hourly observations
- **Features:** 37 engineered features with leakage prevention
- **Models:** 9 models spanning linear to deep learning, with automated champion selection
- **Explainability:** SHAP + LIME + Temporal Grad-CAM for full model transparency
- **Automation:** Hourly ingestion + daily retraining via GitHub Actions
- **Infrastructure:** Serverless AWS deployment managed by Terraform
- **Quality:** Comprehensive EDA (19 analyses), test suite, LangGraph infra validation

12.2 Future Work

1. Expand to multiple cities across Pakistan
2. Integrate satellite imagery (MODIS AOD) for spatial context
3. Implement online learning for model adaptation without full retraining
4. Add ensemble stacking (combining top-3 models)
5. Deploy mobile push notifications for hazardous alerts

Appendix A

Environment Variables

Table A.1: Complete Environment Variable Reference

Variable	Default	Description
AQICN_API_KEY	demo	AQICN API token
OPENWEATHER_API_KEY	–	OpenWeatherMap key
CLEARML_API_ACCESS_KEY	–	ClearML access key
CLEARML_API_SECRET_KEY	–	ClearML secret key
TARGET_CITY	Sargodha	Forecast target
FORECAST_HORIZON_HOURS	72	Prediction window
LOOKBACK_WINDOW_HOURS	72	Input sequence length
AQI_ALERT_THRESHOLD	150	Alert trigger level
LSTM_HIDDEN_SIZE	128	LSTM hidden dim
LSTM_NUM_LAYERS	2	LSTM layers
OPTUNA_N_TRIALS	50	HPO budget
LOG_LEVEL	INFO	Logging verbosity

Appendix B

Project File Structure

Listing B.1: Complete project structure

```
1 .
2 +-- config/           Configuration (Pydantic BaseSettings)
3 +-- eda/              Exploratory Data Analysis (19 analyses)
4 +-- data_pipeline/    Data ingestion & feature engineering
5 +-- feature_pipeline/ ClearML Feature Store management
6 +-- training_pipeline/ Model training, evaluation,
   explainability
7 |   +-- models/       9 model implementations
8 +-- deployment/
9 |   +-- api/          Flask REST API
10 |  +-- streamlit_app/ Streamlit dashboard
11 +-- infrastructure/
12 |   +-- terraform/    AWS IaC
13 |   +-- validation_agents/ LangGraph multi-agent validation
14 +-- data/            Feature store (Parquet) + backfill CSV
15 +-- models/          Trained model artifacts
16 +-- tests/           pytest test suite
17 +-- docs/            This documentation
18 +-- .github/workflows/ CI/CD (hourly + daily + PR)
19 +-- Dockerfile        Multi-stage build
20 +-- docker-compose.yml Container orchestration
21 +-- requirements.txt   Python dependencies
```